Python — Today's Complete Class Notes

Complete notes on dictionaries, enumerate(), zip(), list-to-dictionary conversion, dictionary methods, and registration/login programs.

1. Dictionary Basics

A dictionary stores data as **key-value pairs** using curly braces { }.

```
student = {"name": "Mahesh", "age": 24, "course": "Python"}
print(student["name"])
# Mahesh
```

• **Key:** identifier used to access data.

• **Value:** data stored against a key.

• **Item:** one complete key-value pair, such as ('name', 'Mahesh').

2. enumerate()

enumerate() gives both the index and the value while looping through a sequence.

```
k = [11, 12, 13, 14]
print(list(enumerate(k)))
# [(0, 11), (1, 12), (2, 13), (3, 14)]

for i, j in enumerate(k):
    print(i, j)
```

Here i is the index and j is the value. Use enumerate(k, start=1) to start numbering at 1.

3. zip()

zip() combines corresponding elements from two or more sequences.

```
key = ["a", "b", "c", "d"]
val = [31, 32, 33, 34]

print(list(zip(key, val)))
# [('a', 31), ('b', 32), ('c', 33), ('d', 34)]
```

4. Convert One List to a Dictionary

```
k = [11, 12, 13, 14]
d = dict(enumerate(k))
print(d)
# {0: 11, 1: 12, 2: 13, 3: 14}
```

The longer approach is:

```
d = {}
for i, j in enumerate(k):
    d[i] = j
print(d)
```

5. Convert Two Lists to a Dictionary

```
key = ["a", "b", "c", "d"]
val = [31, 32, 33, 34]

d = dict(zip(key, val))
print(d)
# {"a": 31, "b": 32, "c": 33, "d": 34}
```

Remember: dict(zip(list1, list2)) means two lists → dictionary.

6. Dictionary Methods — Quick Reference

Method	Meaning
`clear()`	Remove all items
`copy()`	Create a copy
`fromkeys()`	Create dictionary from keys
`get()`	Get a value safely
`items()`	Get key-value pairs
`keys()`	Get keys
`pop()`	Remove a specified key
`popitem()`	Remove last inserted item
`setdefault()`	Get existing value or create default
`update()`	Add or modify data
`values()`	Get values

7. clear()

Removes all items; the dictionary remains but becomes empty.

```
d = {"a": 10, "b": 20}
d.clear()
print(d)
# {}
```

8. copy()

Creates a copy of the dictionary.

```
d = {"name": "Mahesh", "age": 24}
c = d.copy()
c["age"] = 25
print(d)  # {"name": "Mahesh", "age": 24}
print(c)  # {"name": "Mahesh", "age": 25}
```

9. fromkeys()

Creates a new dictionary from a sequence of keys. Without a value, values are None.

```
keys = ["name", "age", "course"]
print(dict.fromkeys(keys))
# {"name": None, "age": None, "course": None}

print(dict.fromkeys(keys, "Unknown"))
# {"name": "Unknown", "age": "Unknown", "course": "Unknown"}
```

10. get()

Returns the value for a key. If missing, it returns None or your supplied default.

```
d = {"name": "Mahesh", "age": 24}
print(d.get("name"))                 # Mahesh
print(d.get("city"))                 # None
print(d.get("city", "Not Available")) # Not Available
```

11. items()

Returns key-value pairs and is very useful in loops.

```
d = {"name": "Mahesh", "age": 24}
```

```
print(list(d.items()))
# [("name", "Mahesh"), ("age", 24)]

for key, value in d.items():
    print(key, value)
```

12. keys()

Returns all dictionary keys.

```
d = {"name": "Mahesh", "age": 24}
print(list(d.keys()))
# ["name", "age"]
```

13. values()

Returns all dictionary values.

```
d = {"name": "Mahesh", "age": 24}
print(list(d.values()))
# ["Mahesh", 24]
```

14. pop()

Removes a specified key and returns its value.

```
d = {"name": "Mahesh", "age": 24}
x = d.pop("age")
print(x)  # 24
print(d)  # {"name": "Mahesh"}
```

15. popitem()

Removes and returns the last inserted key-value pair.

```
d = {"name": "Mahesh", "age": 24, "course": "Python"}
x = d.popitem()
print(x)  # ("course", "Python")
print(d)  # {"name": "Mahesh", "age": 24}
```

16. setdefault()

If the key exists, it returns the existing value. If it does not exist, it creates the key with the default value.

```
d = {"name": "Mahesh"}

print(d.setdefault("name", "Ravi"))
# Mahesh

print(d.setdefault("course", "Python"))
# Python

print(d)
# {"name": "Mahesh", "course": "Python"}
```

17. update()

Adds new key-value pairs or changes existing values.

```
d = {"name": "Mahesh", "age": 24}

d.update({"course": "Python"})
print(d)
# {"name": "Mahesh", "age": 24, "course": "Python"}

d.update({"age": 25})
print(d["age"])
# 25
```

18. Single User Login

```
user = "mahesh"
password = "123"

while True:
    user_name = input("Enter user id: ")
    user_password = input("Enter password: ")

    if user_name == user and user_password == password:
        print("Success")
        break
    elif user_name == user and user_password != password:
        print("Enter correct password")
    else:
        print("Provide correct username")
```

19. Multiple Users Using Two Lists

Two lists can store usernames and passwords at matching indexes.

```
user = ["test", "test2", "test3", "test4"]
password = ["123", "456", "678", "890"]

while True:
    user_name = input("Enter user id: ")
    user_password = input("Enter password: ")

    if (user_name in user and user_password in password
        and user.index(user_name) == password.index(user_password)):
        print("Success")
        break
    elif user_name in user and user_password not in password:
        print("Enter correct password")
    else:
        print("Provide correct username")
```

20. Registration Using Lists

```
user = []
password = []

j = input("Create the user name: ")
k = input("Create the password: ")

user.append(j)
password.append(k)

print("Thank you for registering")
```

21. Registration + Login Using a Dictionary

A dictionary is cleaner because username can be the key and password can be its value.

```
users = {}

# Registration
username = input("Create username: ")
password = input("Create password: ")
users[username] = password
print("Registration successful")

# Login
login_user = input("Enter username: ")
login_password = input("Enter password: ")

if login_user in users and users[login_user] == login_password:
    print("Login successful")
else:
```

```
    print("Invalid username or password")
```

22. Complete Registration + Login Menu

```
users = {}

while True:
    print("\n1. Register")
    print("2. Login")
    print("3. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        username = input("Create username: ")
        password = input("Create password: ")
        if username in users:
            print("Username already exists")
        else:
            users[username] = password
            print("Registration successful")

    elif choice == "2":
        username = input("Enter username: ")
        password = input("Enter password: ")
        if username in users and users[username] == password:
            print("Login successful")
            break
        else:
            print("Invalid username or password")

    elif choice == "3":
        print("Thank you")
        break
    else:
        print("Invalid choice")
```

23. Quick Revision

• **enumerate()** → index + value

• **zip()** → combine corresponding elements

• **dict(enumerate(list))** → one list to dictionary

• **dict(zip(list1, list2))** → two lists to dictionary

• **keys()** → keys only; **values()** → values only; **items()** → key + value

• **get()** → safely get a value

• **pop()** → remove selected key; **popitem()** → remove last inserted item

• **update()** → add/change; **clear()** → remove all

• **copy()** → copy; **fromkeys()** → create from keys; **setdefault()** → add only if missing

Note: Real applications should not store passwords as plain text. Production systems use secure password hashing and proper authentication.